

Lab 07 VPN with WireGuard

Driving Question

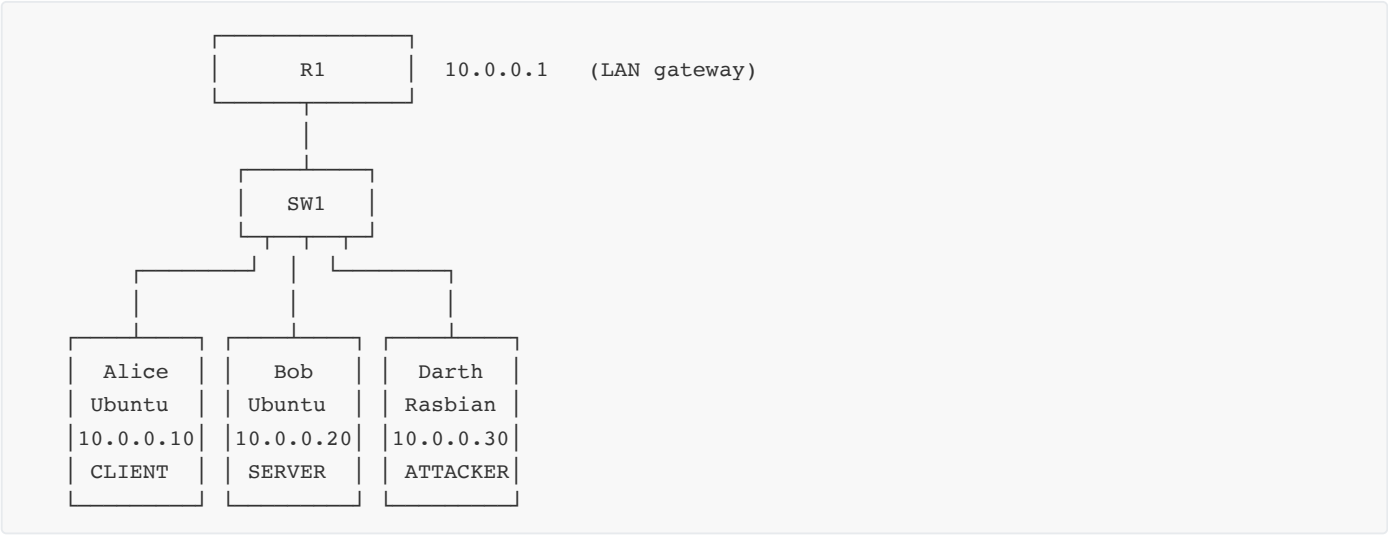
A VPN protects exactly what you route through it. Nothing more, nothing less.

This lab is not about installing WireGuard. It is about discovering, layer by layer, what a VPN does and does not protect — and how the difference between a *split tunnel* and a *full tunnel* is the difference between two completely different security postures.

You will build a tunnel, then peel back its false sense of security from the perspective of Darth, an attacker on the same LAN as Alice and Bob. From ES 01 onward, Darth is *actively between* Alice and Bob, and at each step you will measure what he can still observe even from that privileged position.

Lab Environment

The lab runs entirely inside GNS3. The topology is:



Node	Role	LAN IP	VPN IP
Alice	VPN client	10.0.0.10	10.10.10.2
Bob	VPN server	10.0.0.20	10.10.10.1
Darth	Attacker on the same LAN	10.0.0.30	—
R1	LAN gateway	10.0.0.1	—
SW1	Switch	—	—

Dependencies (already installed on the lab nodes — verify if needed). Each role needs a different set of packages:

On Alice (VPN client):

```
sudo apt install wireguard wireguard-tools dnsutils
```

- `wireguard` + `wireguard-tools` — kernel module and `wg`/`wg-quick` commands (ES 01)
- `dnsutils` — provides `dig` to generate DNS queries (ES 03)

On Bob (VPN server):

```
sudo apt install wireguard wireguard-tools dnsmasq iptables tshark
```

- `wireguard` + `wireguard-tools` — endpoint of the tunnel
- `dnsmasq` — DNS resolver listening inside the tunnel (ES 03)
- `iptables` — NAT/MASQUERADE rules for full-tunnel routing (ES 04)
- `tshark` — used in ES 03 to verify DNS queries arrive on `wg0`

On Darth (attacker):

```
sudo apt install tshark dsniff
```

- `tshark` — capture and analyze traffic
- `dsniff` — provides `arpspoof`, used to position Darth as a man-in-the-middle on the managed switch

Darth Gets in Position (MITM Setup)

The lab topology uses a **managed switch (SW1)**, not a hub. A managed switch learns MAC addresses and forwards unicast frames only to the port that owns the destination MAC. This means Darth — even though he is on the same LAN as Alice and Bob — does **not** see their unicast traffic by default. If you started `tshark` on Darth right now, you would see only broadcast/multicast frames and your own traffic. Nothing of Alice-Bob traffic.

This is exactly the reality of any modern wired network. Passive sniffing on a managed switch does not work. To observe anyone else's traffic, **Darth must first put himself in the path of the packets** — i.e. perform a man-in-the-middle attack.

You already know how. In **Lab 04** you saw CAM-table overflow and you did ARP spoofing. Here you reuse ARP spoofing as a stepping stone: it is the cleanest way to redirect Alice's and Bob's unicast traffic through Darth, without affecting performance of the rest of the LAN.

Threat model implication. Throughout ES 01–04, you will assume Darth has *successfully* completed this MITM step.

Step 1 — Verify Darth sees nothing without MITM (baseline)

On Darth, start a capture on the LAN interface:

```
# On DARTH
sudo tshark -i eth0
```

In another console, generate traffic between Alice and Bob:

```
# On ALICE
ping 10.0.0.20 -c 5
```

Observation: Darth sees nothing of the ICMP exchange. SW1 forwards Alice's frames only to Bob's port. Only ARP broadcasts and the occasional multicast reach Darth.

Stop the capture.

Step 2 — Enable IP forwarding on Darth (MANDATORY)

This step is not optional and not cosmetic. Once you complete Step 3, **every packet** Alice sends to Bob, to R1, or to any address on the LAN will be delivered to Darth instead. If Darth does not forward those packets onward to their real destination, the entire LAN becomes unreachable from Alice's point of view:

- Pings to Bob (10.0.0.20) hang
- DNS queries to R1 (10.0.0.1) time out
- The WireGuard tunnel handshake never completes
- Alice notices immediately and the attack is exposed

Forwarding turns Darth into a **transparent relay**: he sees every packet but also forwards it to where it was meant to go. Victims notice nothing.

```
# On DARTH — enable forwarding
sudo sysctl -w net.ipv4.ip_forward=1

# Verify it is actually 1 — this is the single most common cause of broken labs
sysctl net.ipv4.ip_forward
# expected output: net.ipv4.ip_forward = 1
```

Note on persistence: `sysctl -w` only sets the value until the next reboot.

Reference card:

<code>sudo sysctl -w net.ipv4.ip_forward=1</code>	enable IP forwarding now
<code>sysctl net.ipv4.ip_forward</code>	check current value
<code>cat /proc/sys/net/ipv4/ip_forward</code>	equivalent direct read

Step 3 — Poison ARP on Alice, Bob, and R1

Use `arpspoof` (from the `dsniff` package). **Why four processes and not two?** You will poison two *independent pairs*, each one covering a different category of Alice's traffic:

- **Pair 1 — Alice-Bob (peer poisoning):** intercepts *direct LAN traffic* between Alice and Bob. Needed for ES 01-02 (the WireGuard tunnel itself runs over this pair).
- **Pair 2 — Alice-R1 (gateway poisoning):** intercepts *all traffic Alice sends outside the LAN*.

```
# — Pair 1: Alice <-> Bob (peer poisoning) — needed for ES 01, ES 02 ———
# Tell Alice that 10.0.0.20 (Bob) is at Darth's MAC
sudo arpspoof -i eth0 -t 10.0.0.10 10.0.0.20

# Tell Bob that 10.0.0.10 (Alice) is at Darth's MAC
sudo arpspoof -i eth0 -t 10.0.0.20 10.0.0.10

# — Pair 2: Alice <-> R1 (gateway poisoning) — needed for ES 03, ES 04 ———
# Tell Alice that 10.0.0.1 (R1) is at Darth's MAC
sudo arpspoof -i eth0 -t 10.0.0.10 10.0.0.1

# Tell R1 that 10.0.0.10 (Alice) is at Darth's MAC
sudo arpspoof -i eth0 -t 10.0.0.1 10.0.0.10
```

Think: gateway poisoning is the most powerful single MITM primitive on a LAN. Why? Because the default gateway is the *next-hop* for every destination that is not on the local subnet — Internet, public DNS, every external service. Poisoning one binding (R1) gives Darth visibility on Alice's entire Internet-bound traffic, regardless of what upstream services Alice uses. The poisoning of `10.0.0.1` is what makes Darth see DNS queries to `8.8.8.8` in ES 03 even though Alice never *talks* to R1 directly — Alice's kernel just hands every outbound frame to R1's MAC, which is now Darth's MAC.

Reference card:

<code>arp spoof -i <iface> -t <victim> <impersonated></code>	tell <code>victim</code> that <code>impersonated</code> 's IP is at Darth's MAC
<code>sysctl -w net.ipv4.ip_forward=1</code>	turn Darth into a transparent relay so victims do not notice
<code>jobs</code>	list running background processes
<code>kill %1 %2 %3 %4</code>	stop the four <code>arp spoof</code> processes at the end of the lab

Step 4 — Verify the MITM is active

On Alice, inspect the ARP table for *both* poisoned bindings:

```
# On ALICE
ip neigh show 10.0.0.20    # Bob — should resolve to Darth's MAC
ip neigh show 10.0.0.1    # R1 — should also resolve to Darth's MAC
```

On Darth, find your own MAC for comparison:

```
# On DARTH
ip link show eth0
```

Observation: both `10.0.0.20` and `10.0.0.1` on Alice now point to Darth's MAC. Compare the MAC value with what `ip link show eth0` reports on Darth — they must match. Symmetrically, run `ip neigh show 10.0.0.10` on Bob and on R1; both should also resolve to Darth's MAC.

Step 5 — Confirm Darth now sees Alice's traffic on both pairs

Re-run the capture from Step 1, and generate traffic in **both** directions Alice cares about:

```
# On DARTH
sudo tshark -i eth0 -Y "icmp"
```

```
# On ALICE
ping 10.0.0.20 -c 3    # towards Bob    (needed by ES 01-02)
ping 10.0.0.1 -c 3    # towards R1    (needed by ES 03-04)
```

Observation: Darth now sees every ICMP echo request and reply for **both** flows. He is in the middle of every conversation Alice has on this LAN. **Keep all four `arp spoof` processes running for the rest of the lab** — they are the prerequisite for every subsequent capture in ES 01-04.

Open questions

- ARP spoofing only works because ARP has no authentication. Which Layer-2 defenses from earlier labs would block

Darth here?

- What happens to LAN connectivity if Darth disables IP forwarding after step 3? Why?
- If Alice and Bob noticed unusually high latency after step 3, what would that suggest about Darth's setup?
- Will WireGuard's encryption (built in the next exercise) still protect Alice and Bob *even though* Darth is already in the middle of their conversation? This is the question ES 01-04 will answer.

Exercise 1 — Layer 1: Payload Protection

What this layer adds: the *content* of messages between Alice and Bob becomes unreadable to Darth. That is the *only* thing L1 gives you.

Scenario: Alice and Bob are on the same untrusted LAN (`10.0.0.0/24`). Darth can see all traffic between them. Alice wants to set up a VPN with Bob so that Darth sees only encrypted WireGuard UDP packets — not the actual payload.

Available tools: `wg`, `wg-quick`, `ip`, `tshark`.

Step 1 — Generate key pairs on Alice and Bob

WireGuard uses public/private key cryptography. Each peer generates its own keypair independently — there is no certificate authority.

On **both** Alice and Bob, generate a keypair and save the keys to files.

Think: why does WireGuard not need a certificate authority like TLS/OpenVPN does? What trust model does it use instead?

Reference card:

<code>wg genkey</code>	generate a new private key (prints to stdout)
<code>wg pubkey</code>	derive the public key from a private key (reads stdin)
<code>wg genkey \ tee privatekey \ wg pubkey > publickey</code>	generate and save both keys in one command
<code>cat privatekey / cat publickey</code>	read saved keys

Step 2 — Configure the WireGuard interface on BOB

Create the WireGuard configuration file `/etc/wireguard/wg0.conf` on Bob. The file has two sections:

- `[Interface]` — Bob's own settings: private key, VPN IP, listening port
- `[Peer]` — Alice's settings: Alice's public key, allowed IPs

Use VPN subnet `10.10.10.0/24`: Bob gets `10.10.10.1/24`, Alice gets `10.10.10.2/24`.

Think: what does `AllowedIPs` mean in WireGuard? It controls both routing and which packets are accepted from this peer.

Reference card:

```
[Interface]
PrivateKey = <bob-private-key>
Address = 10.10.10.1/24
ListenPort = 51820

[Peer]
PublicKey = <alice-public-key>
AllowedIPs = 10.10.10.2/32
```

Step 3 — Configure the WireGuard interface on ALICE

Create `/etc/wireguard/wg0.conf` on Alice. Same structure, but:

- `[Interface]` uses Alice's private key and IP (`10.10.10.2/24`)
- `[Peer]` references Bob: Bob's public key, Bob's LAN IP and port (`Endpoint = 10.0.0.20:51820`), allowed IPs

Reference card:

```
[Interface]
PrivateKey = <alice-private-key>
Address = 10.10.10.2/24

[Peer]
PublicKey = <bob-public-key>
Endpoint = 10.0.0.20:51820
AllowedIPs = 10.10.10.1/32
PersistentKeepalive = 25
```

Step 4 — Bring up the tunnel on both nodes

Start WireGuard on both sides. Bring up Bob first.

Reference card:

<code>sudo wg-quick up wg0</code>	bring up the WireGuard interface
<code>sudo wg-quick down wg0</code>	tear down the interface
<code>sudo wg show</code>	show tunnel status, peers, and traffic counters
<code>ip addr show wg0</code>	verify the VPN interface has the correct IP

Step 5 — Verify the tunnel is working

From Alice, ping Bob's VPN IP:

```
ping 10.10.10.1
```

On Bob, run `sudo wg show` — you should see Alice listed as a peer with a recent handshake timestamp and non-zero traffic counters.

Step 6 — Verify encryption from Darth's perspective

On Darth, capture WireGuard traffic on the LAN:

```
sudo tshark -i eth0 -Y "udp.port == 51820" -T fields -e ip.src -e ip.dst -e udp.length -c 10
```

You should see UDP packets between 10.0.0.10 and 10.0.0.20 — no ICMP, no readable payload.

Open questions

- What does the WireGuard handshake look like on the wire? Run tshark on Darth while bringing up the tunnel — what protocol and port do you see?
- WireGuard uses UDP, not TCP. What are the implications for reliability and NAT traversal?
- What happens if you put the wrong public key in the [Peer] section? Does the tunnel silently fail or does WireGuard give an error?

L1 Wrap-Up — Update the Protection Table

What Darth NO longer sees	What Darth STILL sees
Content of Alice↔Bob messages	Source/destination IPs (10.0.0.10, 10.0.0.20)
Inner protocol (ICMP, HTTP, ...)	Traffic volume and timing
	The fact that WireGuard is in use (UDP/51820)
	Anything NOT destined for 10.10.10.1

L1 gives you payload encryption. The right column shows everything still exposed. ES 02 measures it; ES 03 and ES 04 fix it.

Exercise 2 — Layer 2: What L1 Does NOT Protect

What this exercise adds: *no new protection*. Its purpose is to **measure** how much of Alice's communication is *still readable* to Darth even with L1 fully active. You will internalize, by direct observation, the difference between cryptographic confidentiality (which L1 gives) and anonymity (which it does not).

Scenario: compare what Darth sees with and without the VPN active.

Step 1 — Capture traffic without VPN

Tear down the tunnel on Alice. On Darth, start capturing while Alice sends traffic to Bob.

```
# On ALICE
sudo wg-quick down wg0

# On DARTH
sudo tshark -i eth0 -w /tmp/no_vpn.pcap
```

From Alice, send some traffic:

```
ping 10.0.0.20 -c 5
```

Stop the capture on Darth (Ctrl+C).

Step 2 — Re-enable the VPN and repeat

```
# On BOB then ALICE
sudo wg-quick up wg0

# On DARTH
sudo tshark -i eth0 -w /tmp/with_vpn.pcap
```

Send the same traffic from Alice, but now through the tunnel:

```
ping 10.10.10.1 -c 5
```

Stop the capture.

Step 3 — Compare the two captures

```
# On DARTH
echo "=== WITHOUT VPN ===" && tshark -r /tmp/no_vpn.pcap -T fields -e ip.src -e ip.dst -e
_ws.col.Protocol | head -20
echo "=== WITH VPN ===" && tshark -r /tmp/with_vpn.pcap -T fields -e ip.src -e ip.dst -e
_ws.col.Protocol | head -20
```

Reference card:

<pre>tshark -r <file> -T fields -e ip.src -e ip.dst -e _ws.col.Protocol</pre>	show IPs and protocols from a saved capture
<pre>tshark -r <file> -Y "udp.port == 51820"</pre>	filter WireGuard traffic by port

Open questions

- In the VPN capture, what does Darth see? Can he tell Alice and Bob are communicating?
- Can Darth determine that WireGuard is being used? How?
- What metadata remains visible even inside a WireGuard tunnel?
- How does WireGuard compare to the SSH tunnel from previous editions of this lab in terms of what Darth observes?

L2 Wrap-Up — Reflection

Darth has captured 5 minutes of Alice's traffic. Even without reading the content, what can he infer about her?

Write down at least three things he can still learn (hint: who she talks to, when, how much, what *kind* of activity — a long download has a recognizably different pattern than a chat or a video call). This is **traffic analysis**, and it is what ISPs, governments, and ad networks rely on in practice. **The payload is often not what matters.**

L1 covers content. L2 tells us content was only part of the problem.

Exercise 3 — Layer 3: DNS

What this layer adds: DNS queries get routed *through* the tunnel. This is your first concrete example of the lab's central principle — *the VPN protects what you route through it*. To protect DNS we will literally tell the OS "send DNS to a resolver that lives inside the tunnel".

Aside — why Darth sees DNS even though Alice does not talk to R1 directly. On Ubuntu, `/etc/resolv.conf` typically points to `127.0.0.53` (systemd-resolved stub). The *real* upstream resolver is configured per-link — run `resolvectl status` on Alice to see it. It is often a public resolver like `8.8.8.8` or `1.1.1.1`, not the LAN gateway R1. You might expect Darth to see nothing in that case — but he does. Reason: any packet destined to `8.8.8.8` is not on the local subnet, so Alice's kernel sends it via the default gateway (`10.0.0.1`). The frame's *destination MAC* is therefore R1's MAC. After gateway poisoning that MAC is **Darth's MAC**. Darth captures the frame, then forwards it to R1, which forwards it to `8.8.8.8`. From the application's point of view nothing changes — but the packet has traversed Darth's interface, and tshark records it.

Scenario: Alice has WireGuard active and believes her traffic is fully protected. Darth is on the same LAN. Even though the VPN tunnel is up, Darth can still read the domain names Alice is visiting — because DNS queries are leaking outside the tunnel in cleartext.

A **DNS leak** happens when the system resolver sends DNS queries through the physical interface instead of through the VPN tunnel. The tunnel encrypts everything *except* DNS, so Darth sees a stream of domain names even though he cannot read the actual page content.

Available tools: `tshark`, `dig`, `resolvectl`, `wg-quick`, `dnsmasq`.

Step 1 — Reproduce the DNS leak

Make sure the WireGuard tunnel is active but Alice's `wg0.conf` has **no** `DNS =` line in `[Interface]`.

On **Darth**, capture DNS traffic:

```
sudo tshark -i eth0 -Y "dns" -T fields -e ip.src -e ip.dst -e dns.qry.name
```

On **Alice**, resolve a few domains:

```
dig google.com
dig github.com
dig unical.it
```

What does Darth see? The domain names appear in cleartext — even though WireGuard is running.

Step 2 — Confirm the DNS queries are NOT going through the tunnel

On Alice, check which DNS server is being used:

```
resolvectl status
# or
cat /etc/resolv.conf
```

Think: the DNS server shown is probably R1 (the LAN gateway), not Bob (the VPN server). Why does this happen? Where does the query go before it reaches that resolver?

Reference card:

<code>dig <domain></code>	send a DNS query and show the response
<code>resolvectl status</code>	show current DNS resolver configuration
<code>cat /etc/resolv.conf</code>	show the configured nameserver
<code>tshark -i eth0 -Y "dns" -T fields -e dns.qry.name</code>	capture DNS queries and show domain names

Step 3 — Fix the leak: run a resolver inside the tunnel

The fix has two parts:

- **On Bob:** run a real DNS resolver (`dnsmasq`) that listens on Bob's VPN IP `10.10.10.1`. Any query reaching this address will be answered by Bob after asking a public upstream (`8.8.8.8`).
- **On Alice:** send queries explicitly to `10.10.10.1`. Because that IP is the other end of the tunnel, the query travels inside the WireGuard UDP stream and is invisible to Darth on the LAN.

The "magic" `DNS = 10.10.10.1` line that you can put inside `wg0.conf` only changes Alice's *default* resolver via `resolvconf/openresolv` — a system-integration detail that varies wildly between Linux distributions. The didactic point of this exercise is the *routing* of the query, not the OS configuration. We will therefore use **explicit** `dig @10.10.10.1` queries to demonstrate the principle cleanly; the production-grade automation is documented at the end of the section.

Part 1 — Set up `dnsmasq` on Bob

Three obstacles to handle on modern Ubuntu:

1. **Port 53 is already taken** by `systemd-resolved` — `dnsmasq` cannot start until you free the port.
2. **Bob needs a real upstream resolver** to proxy queries to. R1 in the GNS3 topology only routes IP — it does not resolve names. Use a public resolver (`8.8.8.8`) which Bob can reach through R1's NAT.
3. **`dnsmasq` will not pick up `/etc/resolv.conf` reliably on Ubuntu.** Configure the upstream resolver directly in `/etc/dnsmasq.d/lab07.conf` with `no-resolv` + `server=`. This bypasses every quirk of Ubuntu's `resolv.conf` management.

```
# On BOB

# 1. Install dnsmasq (auto-start will fail because port 53 is taken)
sudo apt install dnsmasq

# 2. Stop and disable systemd-resolved to free port 53
sudo systemctl stop systemd-resolved

# 3. Sanity check: Bob can reach a public resolver through R1's NAT
dig google.com @8.8.8.8
# Should return an ANSWER section with a real IP

# 4. Drop-in dnsmasq config — standalone, hardcoded upstream
sudo tee /etc/dnsmasq.d/lab07.conf >/dev/null <<'EOF'
# Lab 07 — Bob's dnsmasq runs standalone:
# - ignore /etc/resolv.conf (no-resolv)
# - use explicit public upstream resolvers
# - listen on loopback AND on the VPN IP so Alice can reach it through wg0
no-resolv
server=8.8.8.8
server=1.1.1.1
```

```
listen-address=127.0.0.1,10.10.10.1
bind-interfaces
EOF

# 5. Bring up the WireGuard tunnel BEFORE starting dnsmasq.
#   With bind-interfaces dnsmasq can only bind to IPs that already exist.
sudo wg-quick up wg0    # skip if already up

# 6. Start dnsmasq
sudo systemctl start dnsmasq
sudo systemctl status dnsmasq    # must show: Active: active (running)

# 7. Sanity check on Bob — both endpoints must respond
dig google.com @127.0.0.1    # loopback
dig google.com @10.10.10.1    # VPN IP (Bob is talking to itself through wg0)
# Both should return the SAME real IP
```

Part 2 — On Alice, query the resolver inside the tunnel

```
# On ALICE — make sure the tunnel is up
sudo wg show
# Should show wg0 with a recent handshake

# End-to-end check: send a DNS query through the tunnel to Bob's resolver
dig google.com @10.10.10.1
# Look at the SERVER line in the response — must read: 10.10.10.1#53
# This proves the query traveled inside wg0 and was answered by Bob.
```

This is the smallest, most explicit way to demonstrate the fix: **the choice of resolver IP decides whether the query leaks or not.**

Reference card:

<code>sudo systemctl stop / disable systemd-resolved</code>	free port 53 on Ubuntu
<code>dig google.com @8.8.8.8</code> (on Bob)	verify Bob can reach Internet DNS
<code>no-resolv</code>	tell dnsmasq to ignore <code>/etc/resolv.conf</code>
<code>server=8.8.8.8</code> / <code>server=1.1.1.1</code>	hardcoded upstream resolvers for dnsmasq
<code>listen-address=127.0.0.1,10.10.10.1</code>	bind dnsmasq to loopback + VPN IP
<code>bind-interfaces</code>	required when using explicit <code>listen-address</code>
<code>sudo wg-quick up wg0</code> (on Bob) before starting dnsmasq	wg0 must exist before bind
<code>dig google.com @127.0.0.1</code> (on Bob)	does dnsmasq proxy correctly?
<code>dig google.com @10.10.10.1</code> (on Alice)	end-to-end check from inside the tunnel

Step 4 — Verify the leak is fixed (THE KEY MEASUREMENT)

This is the moment that closes the arc of the exercise. You will compare three captures:

Setup — three terminals open in parallel:

```
# On DARTH (terminal A) — capture DNS on the LAN
sudo tshark -i eth0 -Y "dns" -T fields -e ip.src -e ip.dst -e dns.qry.name
```

```
# On BOB (terminal B) — capture DNS INSIDE the tunnel
sudo tshark -i wg0 -Y "dns" -T fields -e ip.src -e ip.dst -e dns.qry.name
```

State 1 — Leak still active (default resolver, query escapes the tunnel):

```
# On ALICE
dig google.com
dig github.com
```

- Terminal A (Darth, LAN): **shows the domain names in cleartext**
- Terminal B (Bob, wg0): shows nothing — these queries never entered the tunnel

State 2 — Fix applied (explicit resolver inside the tunnel):

```
# On ALICE
dig google.com @10.10.10.1
dig github.com @10.10.10.1
```

- Terminal A (Darth, LAN): **shows no DNS query** — only opaque UDP/51820 frames
- Terminal B (Bob, wg0): **shows the queries in cleartext**, source `10.10.10.2` (Alice)

The same DNS lookup, sent to two different resolver IPs, takes two completely different paths and gives two completely different visibility profiles to Darth. **Nothing cryptographic changed between State 1 and State 2 — only the destination of the packet.**

Open questions

- A user with WireGuard active thinks they are fully protected. What can Darth still learn about them if DNS is leaking?
- In State 2, who can read Alice's DNS queries? (Hint: not Darth — but check what Bob's `wg0` capture shows.) What does this mean about *who* you trust when you use a VPN?
- Are there scenarios where DNS could still leak even when Alice tries to query `10.10.10.1`? (Hint: what if an application hardcodes `@1.1.1.1`?)
- Why does the "lazy" fix `DNS = 10.10.10.1` in `wg0.conf` *not* always work on Ubuntu? (See the production note below.)

L3 Wrap-Up — Notice the Pattern

The fix worked because Alice sent the query to an IP that the kernel routes through `wg0`. **The VPN protects what you route through it; the OS routes what you ask it to route.** Trust lives in routing decisions, not in cryptography.

Notice the *trust shift* in State 2: Darth no longer sees the domain names, but **Bob does** — they are perfectly readable on his `wg0` capture. The DNS is no longer "leaked to the LAN", it is "delegated to the VPN provider". You have not eliminated the observer — you have replaced one observer with another, and chosen who you trust.

At this point L1, L2 (measured) and L3 are covered. Everything *else* — LAN traffic, the gateway, any future Internet traffic — is still in the clear. ES 04 closes the gap with one configuration change that generalizes the principle.

Production note — automating the fix

In a real VPN setup you would not type `@10.10.10.1` every time. You would add `DNS = 10.10.10.1` to the `[Interface]` section of Alice's `wg0.conf`, and `wg-quick` would call `resolvconf` at tunnel up to update `/etc/resolv.conf` automatically.

Exercise 4 — Layer 4: Split Tunnel vs Full Tunnel

What this layer adds: all *Internet-bound* traffic gets routed through the tunnel. This is the same principle from ES 03 ("VPN protects what you route through it") generalized to "everything routed by default".

Important warning — what `AllowedIPs = 0.0.0.0/0` actually does on Linux. Commercial documentation calls this "full tunnel", but the behaviour of `wg-quick` is more subtle. With `AllowedIPs = 0.0.0.0/0`, `wg-quick` adds policy-routing rules that send **almost all** traffic into the tunnel — but it deliberately leaves the local LAN visible to the kernel's normal routing, otherwise Alice would lose connectivity to Bob himself (the tunnel endpoint sits on the LAN!) and would create a routing loop. The result: traffic to *external* destinations (Internet) goes inside the tunnel; traffic to *local LAN* destinations (R1, Bob's LAN IP, any `10.0.0.0/24` host) **still exits in cleartext**. This is the exact behaviour of commercial VPN clients (NordVPN, Mullvad) and is not a misconfiguration.

You will learn the difference between **split tunnel** (what you have built in ES 01-03) and **full tunnel** (what you build here), what each protects, and the threat models each fits.

Split Tunnel vs "Full" Tunnel — At a Glance

	Split tunnel (ES 01-03 config)	"Full" tunnel (ES 04 config)
AllowedIPs	<code>10.10.10.1/32</code>	<code>0.0.0.0/0</code>
Goes in the tunnel	Only traffic to <code>10.10.10.0/24</code>	All Internet-bound traffic
Stays in cleartext on LAN	LAN, gateway, Internet, DNS	LAN destinations only (R1, Bob's LAN IP, peers in <code>10.0.0.0/24</code>)
Typical use case	Remote employee → corporate network; site-to-site	Consumer privacy VPN (NordVPN, Mullvad); exit-node
Advantage	Performance: local traffic stays local	Privacy: Internet traffic is invisible to LAN observers
Cost	Partial privacy: any non-VPN traffic is in the clear	Trust shift: Bob now sees ALL Internet traffic in plaintext
Typical mistake	Thinking it protects Internet browsing	Thinking it protects local LAN traffic too — it does not

Scenario: you have a working split tunnel from ES 01-03. You will first *measure* what the split tunnel leaks (Step 0), then enable the "full" tunnel and confirm that **Internet traffic** becomes invisible to Darth — but **LAN traffic remains visible**.

Available tools: `wg-quick`, `iptables`, `sysctl`, `ip route`, `tshark`.

Step 0 — Measure What the Split Tunnel Leaks

Before changing anything, **prove to yourself** that the split tunnel from ES 01-03 only protects traffic to `10.10.10.0/24`. On Darth, capture all of Alice's traffic *except* the WireGuard tunnel itself:

```
# On DARTH
sudo tshark -i eth0 -Y "ip.src == 10.0.0.10 and not udp.port == 51820" -c 10
```

From Alice, send three pings — one that goes through the tunnel, two that do not:

```
# On ALICE
ping 10.10.10.1 -c 3    # destination is the VPN IP — INSIDE the tunnel
ping 8.8.8.8 -c 3      # destination is on the Internet — OUTSIDE (split-tunnel leaks)
ping 10.0.0.1 -c 3     # destination is the LAN gateway — OUTSIDE (LAN destination)
```

Observation: Darth sees the second and third pings in cleartext (ICMP echo requests). The first one is invisible to him (encapsulated as UDP/51820). **Record what you observe.** This is your baseline.

Verify with `ip route get` on Alice before changing anything:

```
ip route get 10.10.10.1    # → dev wg0
ip route get 8.8.8.8       # → dev enp2s0 (LAN!) — this is the leak
ip route get 10.0.0.1      # → dev enp2s0 (LAN)
```

After Step 6 you will repeat this measurement and find that **the second ping (to 8.8.8.8) becomes invisible to Darth** — Internet traffic now goes through the tunnel. The **third ping (to 10.0.0.1) will still be visible**, because wg-quick's "full" tunnel does not capture local LAN destinations. This is the L4 lesson: even "full" tunnel has a precise definition of "full".

Step 1 — Enable IP forwarding on BOB

Bob must be allowed to forward packets between `wg0` and `eth0` (which reaches R1 and the rest of the LAN).

```
# On BOB
sudo sysctl -w net.ipv4.ip_forward=1
```

Think: why is IP forwarding disabled by default on a Linux host? What is the security implication of leaving it on permanently?

Reference card:

<code>sysctl net.ipv4.ip_forward</code>	check current value (1 = enabled, 0 = disabled)
<code>sudo sysctl -w net.ipv4.ip_forward=1</code>	enable forwarding for this session
<code>/etc/sysctl.conf</code>	edit and uncomment to persist across reboots

Step 2 — Add NAT (masquerading) on BOB

Bob must rewrite the source IP of forwarded packets so that replies come back to him. This is the same trick a home router does. Add `PostUp` and `PostDown` lines to Bob's `wg0.conf` so wg-quick installs and removes the NAT rules automatically.

Reference card:

```
# In [Interface] on BOB
PostUp = iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE; iptables -A FORWARD -i wg0 -j ACCEPT;
iptables -A FORWARD -o wg0 -j ACCEPT
PostDown = iptables -t nat -D POSTROUTING -o eth0 -j MASQUERADE; iptables -D FORWARD -i wg0 -j ACCEPT;
iptables -D FORWARD -o wg0 -j ACCEPT
```

Think: what would happen if you wrote `PostUp` rules but forgot `PostDown`? Try it and inspect `iptables -t nat -L POSTROUTING` after `wg-quick down`.

Step 3 — Change AllowedIPs on ALICE to capture all traffic

In Alice's `wg0.conf`, change the `[Peer]` section's `AllowedIPs` from `10.10.10.1/32` to `0.0.0.0/0`. This tells the Linux kernel: "route every IPv4 destination through wg0".

```
# ALICE — [Peer] section
AllowedIPs = 0.0.0.0/0
```

Step 4 — Restart the tunnel on both sides

```
sudo wg-quick down wg0 && sudo wg-quick up wg0
```

Step 5 — Verify routing on ALICE (and understand what changed)

After `wg-quick up`, `ip route show` will look almost identical to before. The default route still says `via 10.0.0.1 dev enp2s0`. This is *expected* — `wg-quick` does not modify the main table. Instead it installs **routing policies** (ip rules) and a **separate routing table** (`51820`) that override the main table for most destinations.

To see what *actually* happens, ask the kernel where it would send a specific packet:

```
# On ALICE
ip rule show
ip route show table 51820

ip route get 10.10.10.1      # → dev wg0      (VPN IP — always in tunnel)
ip route get 8.8.8.8        # → dev wg0      (Internet — now in tunnel)
ip route get 1.1.1.1        # → dev wg0      (Internet — now in tunnel)
ip route get 10.0.0.1       # → dev enp2s0 (LAN — STILL in cleartext!)
ip route get 10.0.0.20      # → dev enp2s0 (Bob's LAN IP — must stay direct, see warning above)
```

Read the output carefully: the kernel sends Internet destinations through `wg0`, but anything in `10.0.0.0/24` (R1, Bob's LAN IP, Darth itself) still goes out `enp2s0` in cleartext. This is the "Full" tunnel's documented limit. **It is the entire point of this exercise.**

Step 6 — Reach an Internet host through the tunnel

`8.8.8.8` is not on Alice's LAN. With the full tunnel active, the packet now travels Alice → `wg0` → encrypted → Bob → MASQUERADE → R1 → Internet → `8.8.8.8`.

```
# On ALICE
ping 8.8.8.8 -c 3
```

(R1 in the GNS3 topology has a NAT cloud reaching the Internet, so Bob can forward Alice's packet on her behalf.)

Step 7 — Repeat the Step 0 Capture

This closes the arc of the whole lab. Run **the exact same capture from Step 0**:

```
# On DARTH — identical to Step 0
sudo tshark -i eth0 -Y "ip.src == 10.0.0.10 and not udp.port == 51820" -c 10
```

```
# On ALICE — exactly the same three pings as Step 0
ping 10.10.10.1 -c 3
ping 8.8.8.8 -c 3
ping 10.0.0.1 -c 3
```

The result — three distinct outcomes for three destinations:

Ping target	Before (split tunnel)	After (full tunnel)
10.10.10.1 (VPN IP)	Invisible to Darth	Invisible to Darth
8.8.8.8 (Internet)	Visible to Darth in cleartext	Invisible to Darth — now inside the tunnel
10.0.0.1 (LAN gateway)	Visible to Darth in cleartext	Still visible — LAN destinations stay direct

The middle row is the win of "full" tunnel. The bottom row is its limit.

Also capture on Bob's outgoing side to see *where* the Internet ping came out:

```
# On BOB
sudo tshark -i eth0 -Y "icmp and ip.dst == 8.8.8.8" -c 5
```

The ICMP packet appears with source IP 10.0.0.20 (Bob), not 10.0.0.10 (Alice). NAT has rewritten the source. From 8.8.8.8's perspective, the ping came from Bob.

Reference card:

ip rule show	show routing policy rules — which table a packet goes to
ip route show table 51820	show wg-quick's dedicated routing table
ip route get <ip>	ask the kernel exactly which interface a packet to <ip> would use
tshark -i eth0 -Y "ip.src == 10.0.0.10 and not udp.port == 51820"	what Darth still sees from Alice outside the tunnel
tshark -i eth0 -Y "icmp and ip.dst == 8.8.8.8" (on Bob)	NATted Internet traffic exiting Bob

Open questions

- Bob's MASQUERADE rule rewrites source IPs of forwarded packets. What happens to reply packets — how does the kernel know to send them back to the right client?
- Suggest an MTU value to add to Alice's [Interface] and explain why it might be needed.
- In a commercial VPN scenario (NordVPN, Mullvad), what does the VPN provider see that an external website cannot? What does that imply about trust?
- The 2023-2024 edition of this course used an SSH point-to-point tunnel (ssh -w 0:0) instead of WireGuard. What practical problem does TCP-over-TCP cause that WireGuard avoids by using UDP?

L4 Wrap-Up — Choose the Right Tunnel for the Scenario

Match each scenario to the correct tunnel mode and justify your choice in one sentence (focus on *what must be protected* and *who is the attacker*):

Scenario	Split / Full / Full + kill switch?
Remote employee accessing the corporate intranet from home	?
Traveler in a coffee shop Wi-Fi wanting privacy from the local network	?
Connecting two branch offices to share a private file server	?
Bypassing geographic content restrictions for streaming	?
Exfiltrating data from a monitored hostile network without alerting the local IDS	?
Lab or test setup where only the encrypted endpoint matters	?

► Suggested answers (justify each one in your own words before reading)

Final Synthesis — Update the Protection Table One Last Time

Layer	What Darth sees	Trust
No VPN	Everything	None needed
L1 + L2 (split tunnel)	UDP/51820, plus all non-tunnel traffic in cleartext	Alice trusts no one beyond herself
L1 + L2 + L3 (split + DNS fix)	UDP/51820, plus all non-tunnel traffic except DNS	Alice trusts Bob with her DNS
L1 + L2 + L3 + L4 ("full" tunnel)	UDP/51820, plus LAN-only traffic (R1, peers in 10.0.0.0/24)	Alice trusts Bob with her Internet traffic

The closing insight: a VPN is not a force field. It is **a routing decision wrapped in cryptography**. Even the so-called "full" tunnel is *not* truly full: it deliberately leaves the local subnet in cleartext so the tunnel itself can function. Choosing split or full is not a technical detail — it is a statement about which threat model you are defending against and which party (your ISP, your local Wi-Fi, your VPN provider, your government) you have decided to trust.

A real "everything in the tunnel" — the kill switch. What if you want to protect even LAN traffic, e.g. when the LAN itself is hostile? You would have to (1) add `iptables/nft` rules on Alice that drop any non-`wg0` outbound traffic, and (2) accept that if the tunnel goes down, Alice is offline. This is called a **kill switch** in commercial VPN parlance. It is a hardening step *on top of* L4, not part of it — and it is what production deployments add when the threat model includes the local network.

Further Reading

- WireGuard whitepaper: <https://www.wireguard.com/papers/wireguard.pdf>
- WireGuard quick start: <https://www.wireguard.com/quickstart/>
- WireGuard vs OpenVPN performance: search "WireGuard OpenVPN benchmark"
- DNS leak test: search "dnsleaktest.com"
- `man wg` and `man wg-quick`
- TCP-over-TCP meltdown: search "Why TCP Over TCP Is A Bad Idea" (Olaf Titz)